

ANUSHKA

Python Backend | ML Systems | Computer Vision

Kanpur, Uttar Pradesh, India • anuushka27@gmail.com • github.com/anuushka-dev • linkedin.com/in/anushka-s-ba1060428

PROFILE

Computer Science (AI/ML) student who ships production-grade systems, not just coursework: a routing and dispatch platform validated end-to-end with 610 automated tests and 480/480 successful load requests, and an intrusion-detection pipeline achieving 99.9% test accuracy on CICIDS2017. Works across Python backend engineering, graph optimization, machine-learning inference, network monitoring, and computer vision, with a consistent emphasis on modular architecture, algorithmic correctness, and evidence-backed performance validation.

EDUCATION

B.Tech in Computer Science (AI/ML)

2023–2027

Dr. A.P.J. Abdul Kalam Technical University

TECHNICAL SKILLS

Languages Python, TypeScript

Backend & APIs FastAPI, Uvicorn, REST APIs, WebSockets, Pydantic

Algorithms & Optimization A*, Bidirectional A*, Dijkstra, Source-Dijkstra, Hungarian Algorithm, Vehicle Routing Problem (VRP), 2-Opt, Large Neighborhood Search (LNS), Distance Matrices

Machine Learning PyTorch, TorchVision, TensorFlow, XGBoost, scikit-learn, EfficientNet-B0, ResNet50, CNNs

Computer Vision & Data OpenCV, MediaPipe, NumPy, Pandas, Pillow, Joblib, Matplotlib, Seaborn

Infrastructure & Reliability Redis, Docker, Docker Compose, Prometheus, SQLite, Scapy, Linux, Health Checks, Request Timeouts, Concurrency Control, Fault Tolerance

Frontend & Engineering React, Vite, React Leaflet, Leaflet, Tailwind CSS, Pytest, Integration Testing, Benchmarking, Git

INDEPENDENT SOFTWARE DEVELOPMENT

2024–Present

- Design and implement modular Python systems separating API interfaces, business logic, inference, routing, optimization, caching, middleware, and infrastructure.
- Build and validate end-to-end services spanning graph algorithms, network-flow machine learning, multimodal computer vision, and real-time inference.

FLAGSHIP PROJECT

CityRoute — Route Optimization & Dispatch Platform

[GitHub](#)

Stack: Python, FastAPI, Uvicorn, OSMnx, NetworkX, Redis, Docker, Docker Compose, Prometheus, Pytest, React, TypeScript, Vite, React Leaflet, Leaflet, Tailwind CSS

Focus: A*, Bidirectional A*, Source-Dijkstra, graph snapping, distance matrices, Vehicle Routing Problem (VRP), 2-Opt, LNS, Hungarian assignment, caching, concurrency, observability, resilience

- Built routing over a 12,969-node / 34,996-edge directed OpenStreetMap graph using A*, Bidirectional A*, and Source-Dijkstra, with graph-node snapping, shortest paths, distance matrices, and road-cost computation.
- Implemented greedy VRP, 2-Opt, Large Neighborhood Search (LNS), and Hungarian assignment for route improvement, multi-order delivery optimization, and driver dispatch.
- Engineered Redis caching and recovery, bounded concurrency, request-timeout enforcement, readiness/liveness checks, lifecycle controls, graceful shutdown handling, and Prometheus reliability metrics.
- Benchmarked the 24-stop open-route workload against real OSM data, measuring a 22.7% median improvement from 2-Opt and a 13.6% median improvement from LNS over greedy.

- Validated road-network dispatch with 80/80 successful Docker cases and load behavior with 480/480 successful requests; exercised correctness, optimization, cache reuse, unreachable-pair handling, overload behavior, and dependency-failure recovery.
- Built a React + TypeScript engineering console using Vite, React Leaflet, Leaflet, and Tailwind CSS; deployed and validated the service on Railway.
- Maintained a 610-test automated suite spanning routing, optimization, caching, middleware, resilience, and API behavior.

ENGINEERING PROJECTS

Network Intrusion Detection System with Physical Context Awareness

[GitHub](#)

Stack: Python, FastAPI, Uvicorn, XGBoost, scikit-learn, Scapy, NumPy, Pandas, Joblib, OpenCV, SQLite, Requests, Matplotlib, Seaborn

Focus: CICIDS2017, 45-feature schema, packet capture, flow construction, ML inference, contextual severity fusion, event persistence, alert delivery

- Built an end-to-end intrusion-detection pipeline spanning packet capture, flow construction, feature extraction, XGBoost inference, contextual severity fusion, event persistence, and alert delivery.
- Trained on CICIDS2017 using a canonical 45-feature schema across 14 classes, with stratified validation and randomized hyperparameter search.
- Achieved approximately 99.9% test accuracy and 0.91 macro-F1, using confusion matrices, classification reports, and class-level performance analysis.
- Implemented live monitoring with Scapy, flow expiration, memory bounds, asynchronous alert processing, retries, failed-batch persistence, structured logging, and FastAPI-based inference.
- Added OpenCV-based physical-context processing and SQLite-backed security-event persistence with a dedicated notification pipeline.

Multimodal Sign Language Translator

[GitHub](#)

Stack: Python, PyTorch, TorchVision, EfficientNet-B0, OpenCV, MediaPipe, FastAPI, WebSockets, Pydantic, Pillow, NumPy, pyttsx3, Streamlit, scikit-learn, Matplotlib, Seaborn

Focus: RGB + hand-skeleton multimodal fusion, CNN branch, temporal stabilization, debouncing, prediction locking, real-time inference, text-to-speech

- Developed a dual-branch multimodal recognition model combining RGB features from EfficientNet-B0 with hand-skeleton features from a dedicated CNN branch, followed by PyTorch feature fusion and classification.
- Built real-time webcam inference with MediaPipe + OpenCV, temporal stabilization, debouncing, prediction locking, sentence construction, and pyttsx3 text-to-speech.
- Exposed live inference through a FastAPI WebSocket backend and an interactive Streamlit application.
- Evaluated model behavior using scikit-learn confusion matrices, classification reports, and class-level precision, recall, and F1 metrics.

Deepfake Image Detection

[GitHub](#)

Stack: Python, TensorFlow, ResNet50

Focus: Dataset cleaning, augmentation, stratified splitting, ROC/PR analysis, learning curves, classification evaluation

- Implemented a ResNet50-based deepfake image classification pipeline with dataset cleaning, augmentation, stratified splitting, and structured preprocessing.
- Evaluated model behavior using confusion matrices, ROC/PR curves, learning curves, and class-level classification metrics.